

ElevateHire: Full Technical Manual

This manual provides a detailed architectural and functional breakdown of the **ElevateHire** application. It is designed for technical stakeholders, developers, and administrators who need to understand the platform's core systems.

🏗️ 1. Architecture Overview: Flutter & Dart

ElevateHire is built on a modern, reactive stack designed for high performance and scalability across Web and Mobile.

What is Flutter?

Flutter is an open-source UI software development kit created by Google. It allows for a single codebase that compiles to **native machine code** for high-end performance.

- **Rendering Engine**: Flutter doesn't use web views; it has its own rendering engine (Skia/Impeller) to draw buttons and layouts directly at 60+ FPS.
- **AOT Compilation**: The application is compiled "Ahead-of-Time," meaning it's extremely fast to start and execute.

The Role of Dart

Dart is the underlying language for Flutter. It is optimized for building user interfaces and features **sound type safety**, ensuring that data flows through the app without hidden errors.

🔒 2. Identity & Security: Supabase Auth & JWT

Security is enforced at two levels: the application (UI) and the database (API).

Supabase Auth (GoTrue)

The platform uses Supabase's authentication service. When a user logs in, Supabase issues a **JSON Web Token (JWT)**.

- **The "AuthWrapper"**: Located at the root of the app, this component listens to a stream called `onAuthStateChange`. If a user is logged in, it shows the dashboard; otherwise, it presents the login screen.

Understanding JWT (The "Secure Pass Card")

Every time the app talks to the database, it sends a **JWT** in the header.

1. **Header**: Defines the algorithm (e.g., HS256).

2. **Payload (The Claims)**: Contains the user's ``id``, ``email``, and ``role``. Supabase uses this to know who is making the request.
3. **Signature**: Ensures that the token hasn't been tampered with.

Row Level Security (RLS)

The database (PostgreSQL) uses **RLS policies**. For example:

- **Student Policy**: ``auth.uid() = id``. This ensures a student can *only* see their own profile and uploaded certificates.
- **College Policy**: Allows admins to view all certificates uploaded by students belonging to their specific organization.

🗣️ 3. ElevateAI: The Hybrid Intelligence Model

ElevateAI serves as a conversational bridge between the user and the platform's features.

Hybrid Strategy

- **Local Rules**: For simple commands like "Go to my profile," the app uses local pattern-matching. This is instantaneous and works without any API cost.
- **Think Mode (Gemini 2.0 Flash)**: When deep career coaching or strategic advice is needed, the app switches to an LLM via **OpenRouter**. It uses the ``google/gemini-2.0-flash-001`` model for its high reasoning capabilities.

Navigation Atlas

The AI contains an internal "Map" of the application. When it identifies a user intent, it returns a special command like ``[NAVIGATE: /student-dashboard?index=5]``, which the Flutter app executes to take the user to the right section.

📄 4. Certification System & Verification

One of the platform's core features is the **Trust-based Certification Pipeline**.

The Flow:

1. **Upload**: Student uploads a PDF/Image.
2. **Extraction**: AI extracts the issuer, logo, and skills.
3. **Verification**:
 - **External**: API checks with providers like NPTEL or Google.
 - **Manual**: College admins review and Approve/Reject.

Rejection Transparency

If a certificate is rejected, the admin provides a **Rejection Reason** (stored in the `rejection_reason` column). This is immediately displayed on the student's dashboard in a high-contrast red alert box.

Public QR Verification

The route `/verify?id=[CERT_ID]` is **publicly accessible**. It uses a premium **Claymorphic (3D)** design theme to wow recruiters. It provides live, server-side validation results without requiring the recruiter to have an account.

📁 5. Recruitment & Job Pipeline

Recruiters use a specialized dashboard to manage the end-to-end hiring cycle.

Job Posting States

Jobs move through a strict lifecycle: `draft` -> `open` -> `closed`.

Recruiters can define precise "Skill Requirements" which the AI uses to rank applicants.

Hiring Pipeline States

Applications for jobs follow a state machine:

- `applied` (Standard entry)
- `shortlisted` (Admin approved)
- `interview` (Scheduled)
- `hired` (Final success)

🚀 6. Infrastructure & Deployment

Vercel Deployment

The web version is hosted on Vercel. We use a custom build process:

```
```bash
flutter build web --release --no-tree-shake-icons
```
```

This ensures all 3D icons and modern typography are bundled correctly for the production web environment.

Supabase BaaS

- **Database**: PostgreSQL (Relational data).
- **Storage**: Used for hosting student CVs and certificate proofs.
- **Real-time**: Used for instant notifications (e.g., when a student is hired or a certificate is approved).

7. Deep-Dive: Supabase Querying & Communication

This section explains how the Flutter application communicates with the Supabase backend and the exact syntax used for data retrieval.

The Singleton Pattern

The application initializes the Supabase client once in `main.dart`:

```
```dart
await Supabase.initialize(url: 'https://...', anonKey: '...');
```
```

Once initialized, any part of the app can access the client via `Supabase.instance.client`. This singleton ensures that we maintain a single connection pool and shared authentication state.

How Querying Works (PostgREST)

Supabase uses **PostgREST**, which automatically turns your Database Schema into a RESTful API. When you write Flutter code, the SDK translates it into a standard HTTP request.

Basic Query Example:

```
```dart
final response = await Supabase.instance.client
 .from('profiles') // 1. Target the table
 .select() // 2. Specify columns (empty = all)
 .eq('id', userId) // 3. Add a filter (Where id = userId)
 .single(); // 4. Expect exactly one record
```
```

Complex Query Example (from `college_repository.dart`):

For advanced filtering, such as the "Approved" tab in the college dashboard, we use a more dynamic approach:

```
```dart
var query = _supabase
 .from('view_college_all_certifications')
 .select()
 .eq('college_id', collegeId);

// Conditional filtering
if (status == 'approved') {
 // Uses the 'in' operator to match any of the successful status strings
 query = query.filter('validation_status', 'in', ['verified', 'approved',
'college_verified']);
}
```

```

}

final response = await query; // Execution happens here
...

The Communication Pipeline
1. Request Construction: The Flutter SDK builds a URL (e.g.,
`.../rest/v1/profiles?id=eq.123`).
2. Auth Injection: The SDK automatically fetches the current JWT (Access
Token) from memory and injects it into the `Authorization: Bearer <TOKEN>`
header.
3. Server-Side RLS: Supabase receives the request, identifies the user from
the JWT, and runs the Row Level Security policies against the query.
4. JSON Response: The database returns the filtered results as a JSON map,
which the app then converts into Dart objects (Models).

Real-time Communication
For features like notifications or live dashboard updates, the app uses
WebSockets:
```dart
final subscription = Supabase.instance.client
  .channel('public:notifications')
  .on(RealtimeListenTypes.postgresChanges,
    ChannelFilter(event: 'INSERT', table: 'notifications'),
    (payload, [ref]) {
      // Handle live update
    })
  .subscribe();
...

```

This avoids constant "polling" and ensures the UI is always in sync with the database.

```

---
```